

LUNCH AND LEARN • FIVE PRACTICES • 60 MINUTES

Agentic engineering

Coding agents are fast. These five practices make them reliable.

THE ART OF THE POSSIBLE

Agents can now carry a ticket all the way to a reviewed pull request.

STEP 01

Plans the change and writes the tests.



STEP 02

Opens the pull request.



STEP 03

Fixes the build until it passes.



STEP 04

Writes an evidence card on how sure it is.



STEP 05

Waits for a person to decide, then writes hand-off notes.

Five practices make that loop trustworthy.

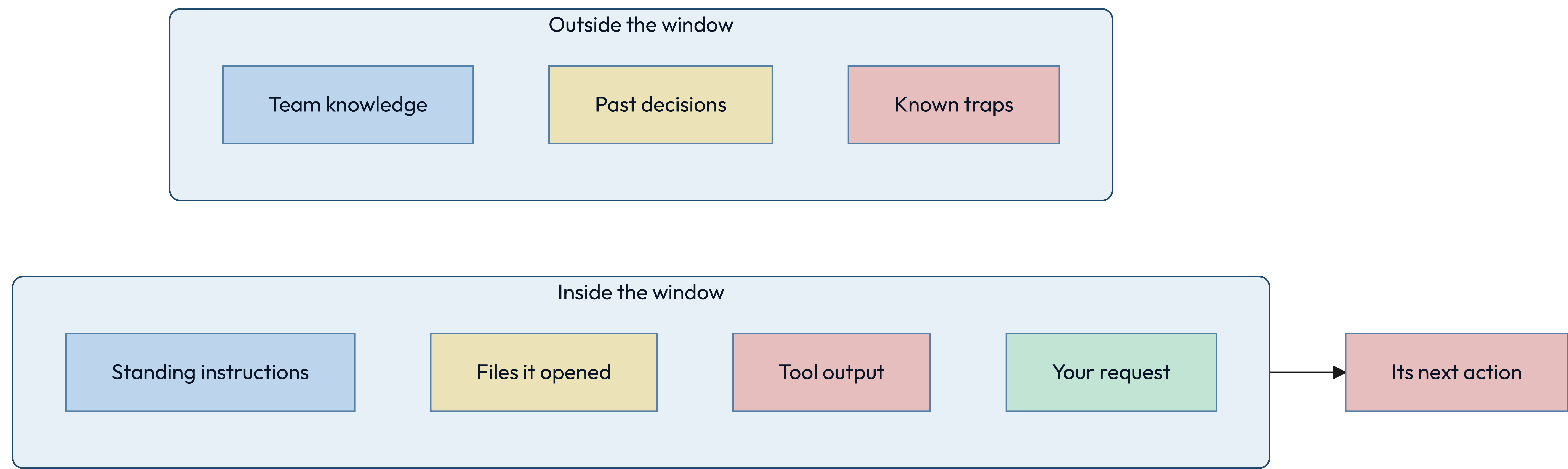
- 01 Context engineering: control what the agent sees
- 02 Autonomy with a reach limit: decide what it may do alone
- 03 Verification you can trust: make every claim prove itself
- 04 A system that learns: turn incidents into rules
- 05 Orchestration: use many agents without losing control

PRACTICE 01

Context engineering

CONTEXT • THE CORE IDEA

Whatever sits outside the context window does not exist for the agent.



Write the always-loaded file as an index, and keep the manual elsewhere.

A MANUAL

Every rule with its full explanation. It grows with each incident, and every session pays to read all of it.



AN INDEX

One line per rule and a pointer to the document that explains it. The agent opens the detail only when the task needs it.

Route the agent to the right document before it starts work.

Read the canonical doc before working in its area

Working on...	Read first	
-----	-----	
Branching, merging, releases	engineering/workflow.md	
Tests, hooks, CI	engineering/development.md	
Something behaving strangely	engineering/gotchas.md	
Building a new script	engineering/capabilities.md	
A past decision	engineering/decisions/	

Four habits keep the window full of signal.

01

Read sections, not whole files

List a document's headings, then open only the part you need.

02

Delegate big reads

A helper agent reads the long log and returns a summary.

03

Quiet the tools

Print failures in full and successes as a single dot.

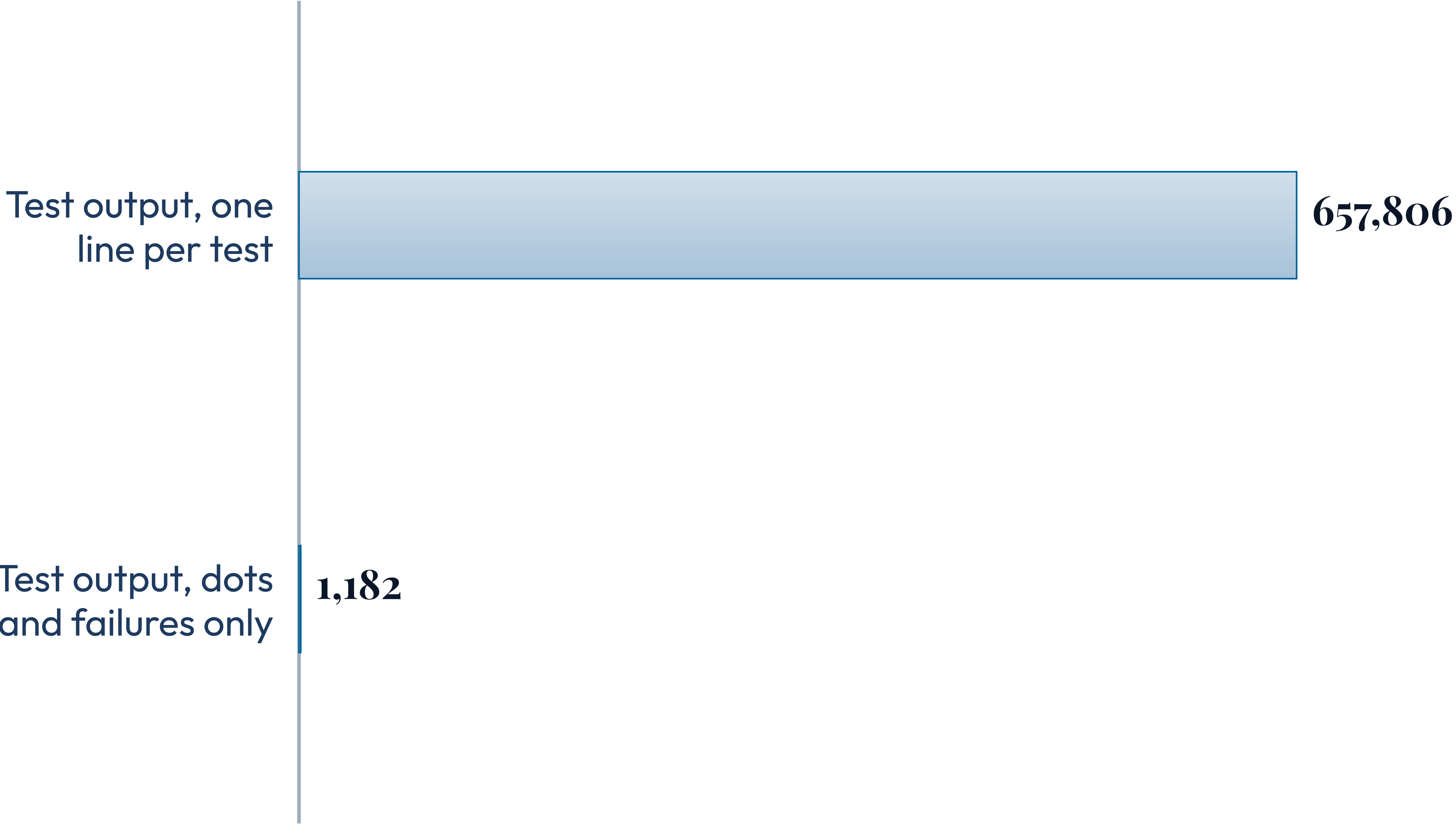
04

Measure the cost

Count tokens from real sessions before you optimize anything.

CONTEXT • WHY QUIET TOOLS MATTER

One quieter test reporter cut a session's biggest cost by 99.8 percent.



Claude Code gives you a control for each of these habits.

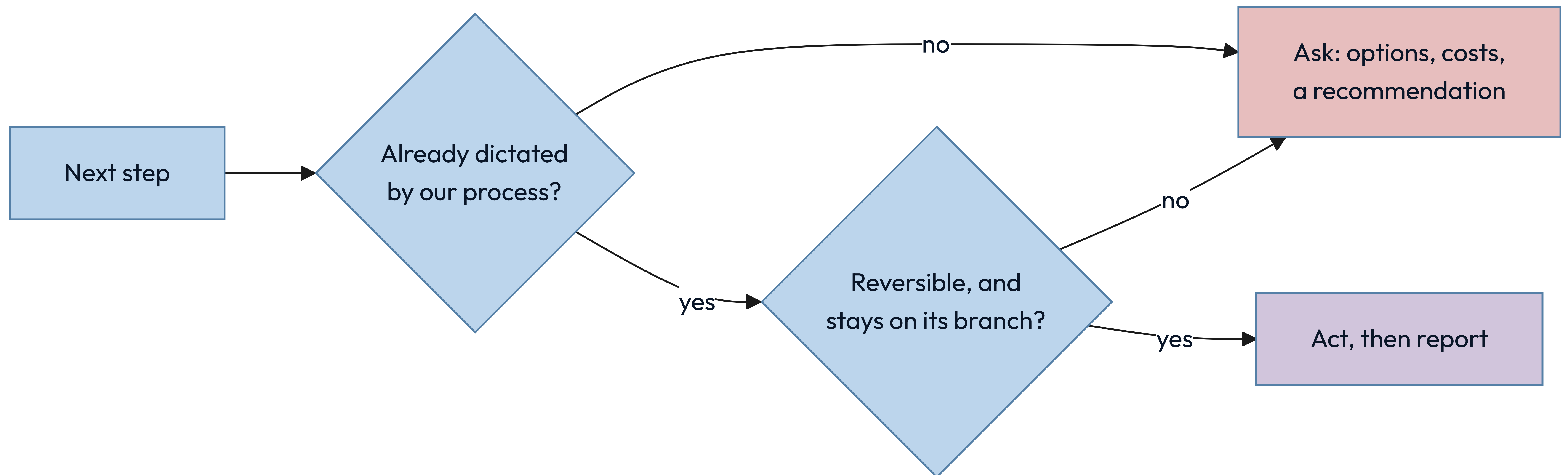
PRACTICE	IN CLAUDE CODE
Standing instructions as an index	<code>CLAUDE.md</code> , with <code>@path</code> imports for detail
Guidance that loads only when relevant	A skill in <code>.claude/skills/<name>/SKILL.md</code>
Big reads outside the main session	A subagent in <code>.claude/agents/</code>
See what fills the window	<code>/context</code> , and <code>/usage</code> for the bill

PRACTICE 02

Autonomy with a reach limit

AUTONOMY · TWO QUESTIONS

Two questions decide whether the agent acts or asks.



Risk depends on reversibility and reach, and difficulty plays no part.

Easy to undo · Stays on the branch.

Code, tests, docs

The agent acts

Easy to undo · Reaches others.

Shared labels, team settings

The agent proposes

Hard to undo · Stays on the branch.

Deleted work, exported files

The agent shows the result first

Hard to undo · Reaches others.

Merges, releases, publishing

A person decides

Five kinds of change always come back to a person.

01

Shared state outside the branch

Labels, boards, settings other people read.

02

The build and test pipeline

Every future change pays for a new step.

03

A number a person set

"About twelve" was a decision, not a suggestion.

04

The meaning of a core document

Rewriting the rules differs from following them.

05

Anything irreversible or public

Merges, releases, comments on someone else's work.

When the agent asks, it brings measured options and a recommendation.

A WEAK QUESTION

"Should I add this check to CI?" The person now has to do the analysis.



A STRONG QUESTION

"Adding it costs 0.52 seconds per run and catches this class of bug. I recommend yes." The person just decides.

Permissions and plan mode turn the reach test into settings.

PRACTICE

IN CLAUDE CODE

Act freely on safe commands, never on risky ones

The allow, ask and deny lists in `.claude/settings.json`

Show the plan before touching anything

Plan mode: `Shift+Tab`

The same limits for every developer

Managed settings, deployed org-wide

PRACTICE 03

Verification you can trust

Agents report what they believe, and belief is not evidence.

WHAT THE AGENT REPORTED

"Verified with a real browser," in the pull request, the commit message and the design note.



WHAT WAS TRUE

The test passed with the fix deleted. It could not fail, so it proved nothing.

VERIFICATION • THE CLAIM RULE

Every "verified" names the surface it ran on and carries proof from it.

01

Name the surface

The real browser, the real export, the real device.

02

Attach the artifact

A screenshot, a log, a measured number from that surface.

03

Say "unverified" out loud

When the real surface is out of reach, write it down.

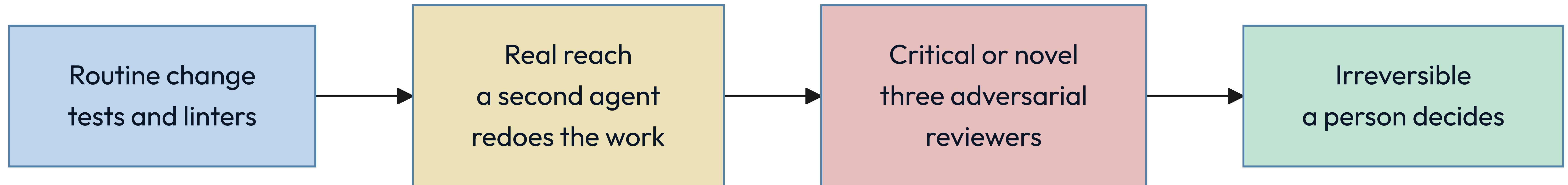
04

Treat "CI is green" as partial

It proves only what CI actually runs.

VERIFICATION • THE LADDER

Scale the checking with the damage a mistake could do.



Three reviewers, each with a different job, find different failures.

Red team

Attacks the change: edge cases, abuse, the input nobody tried.

Inversion

Asks how this could be the wrong answer to the wrong question.

Checker

Re-derives every fact and number from the source.

A check earns trust only by proving it can fail.

Allow zero

The limit is none, and each exception carries a written reason.

Expire exceptions

An exception that no longer matches anything fails the build.

Plant a defect

Each run plants a known bug and fails if the check misses it.

Warn, then block

A new check blocks merges only after a clean track record.

Hooks and subagents make verification automatic.

PRACTICE

IN CLAUDE CODE

Run checks after every edit

A `PostToolUse` hook

Refuse "done" until the tests pass

A `Stop` hook that exits with code 2

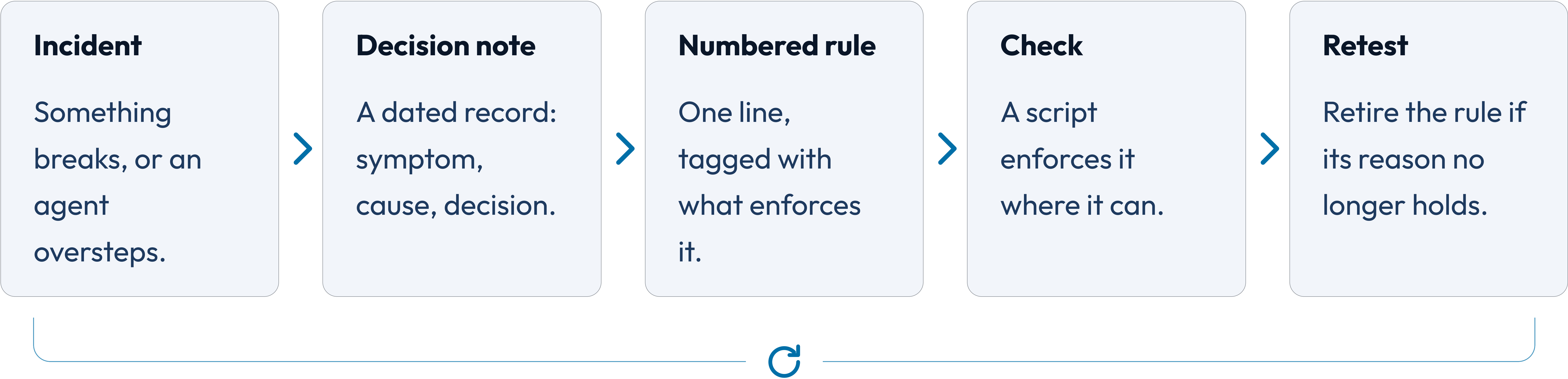
An independent reviewer

A read-only subagent, or `/code-review`

PRACTICE 04

A system that learns

Every rule should travel one loop, from incident to retirement.



A decision note records why, so nobody has to guess later.

```
---  
status: shipped  
summary: Retire the :has() ban; a retest passed 5 of 5  
---
```

Retire rule 12

Symptom	A ban blocked valid CSS for two months
Claim	One browser mishandles :has()
Evidence	Retested on current Chromium: 5 of 5 pass
Decision	Retire the rule; keep its number unused

Proposals, decision records and specs answer different questions.

DOCUMENT	ANSWERS	WHEN IT EXPIRES
Proposal	Which option should we pick?	Once someone decides
Decision record	Why is it this way?	Never; a newer record supersedes it
Spec	What must every implementation do?	Never; you edit it to stay true

Before every merge, the agent grades its own confidence by its weakest point.

Pre-merge: add retry to failed exports

WHAT One automatic retry on a failed export

EVIDENCE Real export run 20 times: 0 failures

RISK Export path only • revert: one commit

UNVERIFIED Safari's download dialog

CONFIDENCE high, set by the evidence axis

raise it by: run the export in Safari

Keep pending work in the repository, one file per item.

IN CHAT

Invisible to the next session. When we finally checked, nearly half of our chat-only to-dos were already done or duplicated.



IN THE REPOSITORY

One small file per item, with a priority and a "done when". Any session can pick it up cold.

PRACTICE 05

Orchestration

ORCHESTRATION · A ROSTER

Give each agent one job and a name, and pick it by its job.

AGENT	ITS ONE JOB
Scout	Find where things live and how they work
Fact-checker	Confirm or refute each claim against the source
CI triage	Find why a check is red, and fix the cause
Red team	Break the change before users do
Checker	Re-derive every number with fresh eyes
Prose checker	Catch unclear or machine-sounding writing

For a wide-open design question, run competing designs and harden only the winner.

STEP 01

**Three to five
design tracks,
each iterating
in one session.**



STEP 02

**One fresh
critic per
track.**



STEP 03

**One shared
fact-checker
across all
tracks.**



STEP 04

**Judges
compare side
by side, and a
person picks.**



STEP 05

**Full
adversarial
review on the
winner only.**

Treat agent count like money: estimate it, cap it, and stop early.

01

Estimate before you launch

Agents and rough tokens, stated up front.

02

Count across the whole session

Ten small fan-outs cost as much as one big one.

03

Ask past about ten agents

A person approves anything bigger.

04

Stop when a round changes nothing

About three rounds is usually enough.

05

Record what it cost

Otherwise nobody learns what it was worth.

Start with one habit per practice, then add a layer each month.

PRACTICE	WEEK 1	MONTH 1	QUARTER 1
Context	☐ A one-page index file	☐ Linked detail docs	☐ Measure what sessions load
Autonomy	☐ Allow and deny lists	☐ A written stop list	☐ Human gate in the platform
Verification	☐ Tests on every change	☐ A check that plants a bug	☐ A second agent on risky work
Learning	☐ A decision log	☐ Pending work as files	☐ Retest your oldest rules

☐ Next step

The practices carry over, but the real surface changes with the work.

WORK	THE REAL SURFACE TO VERIFY	A CHECK THAT PROVES IT CAN FAIL
Data science	A fixed, versioned evaluation set	Plant an input that leaks the answer
Data engineering	The target warehouse	Insert a known-bad row
Analytics and BI	The published dashboard	Plant a mismatch with the source
Services	Staging, or a slice of live traffic	Break a contract on purpose
Tools and CLIs	The built binary's output	A saved expected output that must change

YOUR NEXT STEP

Three moves this week put all five practices in motion.

01

Rewrite your instruction file as an index

One line per rule, a link to the detail.

02

Add one check, then break it on purpose

Watch it fail before you trust it.

03

Start a decision log

One dated note each time something goes wrong.

QUESTIONS

Build the system around the agent